Tarlac State University
COLLEGE OF COMPUTER STUDIES

CASE STUDY IN OPERATING SYSTEMS

Submitted By:
Cabrera, Jacob Stephen S.

Submitted To:
Jo Anne G. Cura

May 15, 2026

1. Introduction

1.1 Overview of CPU Scheduling

CPU Scheduling is one of the fundamental functions of an operating system because it determines how processes are allocated access to the CPU. Since multiple processes may exist simultaneously in the ready queue, the operating system must decide which process should be executed first. CPU scheduling improves overall system performance by maximizing CPU utilization, minimizing waiting time, and improving process responsiveness (Silberschatz, Galvin, & Gagne, 2018).

Scheduling algorithms are generally classified into preemptive and non-preemptive methods. In preemptive scheduling, the operating system may interrupt the currently running process to allow another process to execute. This method improves responsiveness and is commonly used in multitasking systems. In non-preemptive scheduling, a process continues execution until completion before another process is allowed to execute (Stallings, 2018).

Different scheduling algorithms use different approaches for managing process execution. Some algorithms prioritize arrival order while others prioritize burst time or process priority values. These scheduling methods are essential in modern computing systems because they help maintain efficient multitasking and fair resource allocation (Silberschatz et al., 2018).

1.2 Purpose of This Case Study

The purpose of this study is to design and implement a modular CPU Scheduling Simulator using Python programming language. The program demonstrates how common CPU Scheduling Algorithms behave by simulating process execution based on user-defined input values such as Arrival Time, Burst Time, Priority, and Time Quantum.

The study also aims to provide practical understanding of process scheduling concepts by visualizing execution flow through Gantt Charts and computing important scheduling metrics such as Waiting Time and Turnaround Time. Through simulation and comparison, the project demonstrates the strengths and limitations of different scheduling algorithms.

1.3 Objectives of the Program

- To design and develop a CPU Scheduling Simulator
- To implement common CPU Scheduling Algorithms
- To simulate process execution using scheduling techniques
- To compute Waiting Time and Turnaround Time
- To generate Gantt Charts for execution visualization
- To compute Average Waiting Time and Average Turnaround Time
- To compare scheduling algorithms based on performance
- To apply modular programming principles in Python

2. Body

2.1 First-Come, First-Served (FCFS)

First-Come, First-Served (FCFS) is a non-preemptive CPU scheduling algorithm where processes execute according to their arrival order. The first process that enters the ready queue receives CPU access first and continues execution until completion. FCFS is one of the simplest scheduling algorithms because it follows a straightforward queue-based approach (Silberschatz et al., 2018).

The main advantage of FCFS is simplicity and fairness based on arrival order. However, FCFS may produce long waiting times when lengthy processes execute before shorter ones. This problem is commonly known as the convoy effect (Stallings, 2018).

Test Case #1

INPUT

```
Enter number of processes (minimum 3): 3

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 2
Priority: 0

Process 2
Process ID: P2
Arrival Time: 0
Burst Time: 0
Value must be at least 1.
Burst Time: 1
Priority: 0

Process 3
Process ID: P3
Arrival Time: 10
Burst Time: 2
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 1
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P1 | P2 | IDLE | P3 |
0    2    3    10    12

PID     AT      BT      PR      WT      TAT
P1      0       2       0       0       2
P2      0       1       0       2       3
P3      10      2       0       0       2

Average Waiting Time: 0.67
Average Turnaround Time: 2.33

Press Enter to exit...
```

Test Case #2

INPUT

```
Enter number of processes (minimum 3): 5

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 2
Priority: 0

Process 2
Process ID: P2
Arrival Time: 0
Burst Time: 1
Priority: 0

Process 3
Process ID: P3
Arrival Time: 0
Burst Time: 8
Priority: 0

Process 4
Process ID: P4
Arrival Time: 0
Burst Time: 4
Priority: 0

Process 5
Process ID: P5
Arrival Time: 0
Burst Time: 5
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 1
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P1 | P2 | P3 | P4 | P5 |
0    2    3    11    15    20

PID     AT      BT      PR      WT      TAT
P1      0       2       0       0       2
P2      0       1       0       2       3
P3      0       8       0       3       11
P4      0       4       0       11      15
P5      0       5       0       15      20

Average Waiting Time: 6.20
Average Turnaround Time: 10.20

Press Enter to exit...
```

2.2 Shortest Job First (SJF)

Shortest Job First (SJF) is a non-preemptive scheduling algorithm that selects the process with the shortest burst time among all available processes in the ready queue. Once selected, the process executes completely before another process begins execution (Stallings, 2018).

SJF minimizes average waiting time and improves CPU efficiency for shorter processes. However, the algorithm may cause starvation because longer processes may remain waiting indefinitely if short processes continue arriving (Silberschatz et al., 2018).

Test Case #1

INPUT

```
Enter number of processes (minimum 3): 3

Process 1
Process ID: P1
Arrival Time: P
Invalid input. Please enter a valid integer.
Arrival Time: 0
Burst Time: 1
Priority: 0

Process 2
Process ID: P2
Arrival Time: 3
Burst Time: 4
Priority: 0

Process 3
Process ID: P3
Arrival Time: 7
Burst Time: 7
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 2
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P1 | IDLE | P2 | P3 |
0    1    3    7    14

PID     AT      BT      PR      WT      TAT
P1      0       1       0       0       1
P2      3       4       0       0       4
P3      7       7       0       0       7

Average Waiting Time: 0.00
Average Turnaround Time: 4.00

Press Enter to exit...
```

Test Case #2

INPUT

```
Enter number of processes (minimum 3): 5

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 2
Priority: 0

Process 2
Process ID: P2
Arrival Time: 0
Burst Time: 1
Priority: 0

Process 3
Process ID: P3
Arrival Time: 0
Burst Time: 8
Priority: 0

Process 4
Process ID: P4
Arrival Time: 0
Burst Time: 4
Priority: 0

Process 5
Process ID: P5
Arrival Time: 0
Burst Time: 5
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 2
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P2 | P1 | P4 | P5 | P3 |
0    1    3    7    12    20

PID     AT      BT      PR      WT      TAT
P1      0       2       0       1       3
P2      0       1       0       0       1
P3      0       8       0       12      20
P4      0       4       0       3       7
P5      0       5       0       7       12

Average Waiting Time: 4.60
Average Turnaround Time: 8.60

Press Enter to exit...
```

2.3 Shortest Remaining Time (SRT)

Shortest Remaining Time (SRT) is the preemptive version of Shortest Job First scheduling. The CPU is assigned to the process with the shortest remaining burst time. If another process arrives with a smaller remaining burst time, the currently running process is interrupted and replaced (Silberschatz et al., 2018).

SRT improves responsiveness and average waiting time because shorter processes are executed quickly. However, the algorithm increases context switching overhead and is more complex to implement compared to non-preemptive scheduling algorithms (Stallings, 2018).

Test Case #1

INPUT

```
Enter number of processes (minimum 3): 3

Process 1
Process ID: P1
Arrival Time: 1
Burst Time: 2
Priority: 0

Process 2
Process ID: P2
Arrival Time: 1
Burst Time: 3
Priority: 0

Process 3
Process ID: P3
Arrival Time: 2
Burst Time: 3
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 3
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| IDLE | P1 | P2 | P3 |
0    1    3    6    9

PID     AT      BT      PR      WT      TAT
P1      1       2       0       0       2
P2      1       3       0       2       5
P3      2       3       0       4       7

Average Waiting Time: 2.00
Average Turnaround Time: 4.67

Press Enter to exit...
```

Test Case #2

INPUT

```
Enter number of processes (minimum 3): 5

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 2
Priority: 0

Process 2
Process ID: P2
Arrival Time: 1
Burst Time: 1
Priority: 0

Process 3
Process ID: P3
Arrival Time: 2
Burst Time: 8
Priority: 0

Process 4
Process ID: P4
Arrival Time: 4
Burst Time: 4
Priority: 0

Process 5
Process ID: P5
Arrival Time: 5
Burst Time: 5
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 3
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P1 | P2 | P3 | P4 | P5 | P3 |
0    2    3    4    8    13    20

PID     AT      BT      PR      WT      TAT
P1      0       2       0       0       2
P2      1       1       0       1       2
P3      2       8       0       10      18
P4      4       4       0       0       4
P5      5       5       0       3       8

Average Waiting Time: 2.80
Average Turnaround Time: 6.80

Press Enter to exit...
```

2.4 Round Robin (RR)

Round Robin scheduling allocates CPU time equally among processes using a fixed Time Quantum. Each process executes for a limited amount of time before moving back to the ready queue if unfinished. Round Robin is considered a preemptive scheduling algorithm because process execution may be interrupted after the assigned time quantum expires (Stallings, 2018).

Round Robin improves fairness and responsiveness, making it suitable for multitasking and time-sharing systems. However, performance depends heavily on the selected Time Quantum. A very small quantum increases context switching overhead, while a very large quantum causes the algorithm to behave similarly to FCFS (Silberschatz et al., 2018).

Test Case #1

INPUT

```
Enter number of processes (minimum 3): 3

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 3
Priority: 0

Process 2
Process ID: P2
Arrival Time: 10
Burst Time: 10
Priority: 0

Process 3
Process ID: P3
Arrival Time: 4
Burst Time: 12
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 4
Enter Time Quantum: 3
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P1 | IDLE | P3 | P2 | P3 | P2 | P3 | P2 |
0    3    4    10    13    16    19    22    26

PID     AT      BT      PR      WT      TAT
P1      0       3       0       0       3
P2      10      10      0       6       16
P3      4       12      0       6       18

Average Waiting Time: 4.00
Average Turnaround Time: 12.33

Press Enter to exit...
```

Test Case #2

INPUT

```
Enter number of processes (minimum 3): 5

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 2
Priority: 0

Process 2
Process ID: P2
Arrival Time: 0
Burst Time: 1
Priority: 0

Process 3
Process ID: P3
Arrival Time: 0
Burst Time: 8
Priority: 0

Process 4
Process ID: P4
Arrival Time: 0
Burst Time: 4
Priority: 0

Process 5
Process ID: P5
Arrival Time: 0
Burst Time: 5
Priority: 0

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 4
Enter Time Quantum: 2
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P1 | P2 | P3 | P4 | P5 | P3 | P4 | P5 | P3 | P5 | P3 |
0    2    3    5    7    9    11    13    15    17    18    20
```

PID	AT	BT	PR	WT	TAT
P1	0	2	0	0	2
P2	0	1	0	2	3
P3	0	8	0	12	20
P4	0	4	0	9	13
P5	0	5	0	13	18

```
Average Waiting Time: 7.20
Average Turnaround Time: 11.20

Press Enter to exit...
```

2.5 Priority Scheduling

Priority Scheduling assigns CPU access according to process priority values. Processes with higher priority are executed first. In this implementation, lower numerical values represent higher priority. Priority Scheduling may be implemented as either preemptive or non-preemptive depending on system requirements (Stallings, 2018).

Priority Scheduling improves responsiveness for important tasks and provides better system control. However, lower-priority processes may experience starvation if higher-priority processes continuously enter the system (Silberschatz et al., 2018).

Test Case #1

INPUT

```
Enter number of processes (minimum 3): 3

Process 1
Process ID: P1
Arrival Time: 2
Burst Time: 3
Priority: 1

Process 2
Process ID: P2
Arrival Time: 0
Burst Time: 1
Priority: 2

Process 3
Process ID: P3
Arrival Time: 1
Burst Time: 5
Priority: 1

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 5
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P2 | P3 | P1 |
0    1    6    9

PID     AT      BT      PR      WT      TAT
P1      2       3       1       4       7
P2      0       1       2       0       1
P3      1       5       1       0       5

Average Waiting Time: 1.33
Average Turnaround Time: 4.33

Press Enter to exit...
```

Test Case #2

INPUT

```
Enter number of processes (minimum 3): 5

Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 10
Priority: 3

Process 2
Process ID: P2
Arrival Time: 0
Burst Time: 1
Priority: 1

Process 3
Process ID: P3
Arrival Time: 0
Burst Time: 2
Priority: 4

Process 4
Process ID: P4
Arrival Time: 0
Burst Time: 1
Priority: 5

Process 5
Process ID: P5
Arrival Time: 0
Burst Time: 5
Priority: 2

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 5
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| P2 | P5 | P1 | P3 | P4 |
0    1    6    16    18    19

PID     AT      BT      PR      WT      TAT
P1      0       10      3       6       16
P2      0       1       1       0       1
P3      0       2       4       16      18
P4      0       1       5       18      19
P5      0       5       2       1       6

Average Waiting Time: 8.20
Average Turnaround Time: 12.00

Press Enter to exit...
```

2.6 Priority Scheduling with Round Robin

Priority Scheduling with Round Robin combines Priority Scheduling and Round Robin techniques. Processes are first grouped according to priority level, and processes with the same priority share CPU time using Round Robin scheduling with a fixed Time Quantum.

This approach balances fairness and prioritization because higher-priority processes execute first while processes within the same priority level receive equal CPU allocation. However, the algorithm is more complex because it requires both priority management and queue rotation handling (Stallings, 2018).

Test Case #1

INPUT

```
Enter number of processes (minimum 3): 3

Process 1
Process ID: p1
Arrival Time: 2
Burst Time: 3
Priority: 1

Process 2
Process ID: p2
Arrival Time: 0
Burst Time: 1
Priority: 3

Process 3
Process ID: p3
Arrival Time: 2
Burst Time: 2
Priority: 3

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 6
Enter Time Quantum: 2
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| p2 | IDLE | p1 | p3 |
0    1    2    5    7

PID     AT      BT      PR      WT      TAT
p1      2       3       1       0       3
p2      0       1       3       0       1
p3      2       2       3       3       5

Average Waiting Time: 1.00
Average Turnaround Time: 3.00

Press Enter to exit...
```

Test Case #2

INPUT

```
Enter number of processes (minimum 3): 5

Process 1
Process ID: p1
Arrival Time: 0
Burst Time: 4
Priority: 3

Process 2
Process ID: p2
Arrival Time: 0
Burst Time: 5
Priority: 2

Process 3
Process ID: p3
Arrival Time: 0
Burst Time: 8
Priority: 2

Process 4
Process ID: p4
Arrival Time: 0
Burst Time: 7
Priority: 1

Process 5
Process ID: pp5
Arrival Time: 0
Burst Time: 3
Priority: 3

CPU Scheduling Algorithms
1. FCFS
2. SJF
3. SRT
4. Round Robin
5. Priority Scheduling
6. Priority Scheduling with Round Robin
Choose algorithm: 6
Enter Time Quantum: 2
```

PROCESS METRICS AND GANTT CHART

```
Gantt Chart:
| p4 | p2 | p3 | p2 | p3 | p2 | p3 | p1 | pp5 | p1 | pp5 |
0    7    9    11    13    15    16    20    22    24    26    27
```

PID	AT	BT	PR	WT	TAT
p1	0	4	3	22	26
p2	0	5	2	11	16
p3	0	8	2	12	20
p4	0	7	1	0	7
pp5	0	3	3	24	27

```
Average Waiting Time: 13.80
Average Turnaround Time: 19.20

Press Enter to exit...
```

3. Summary/Takeaways

3.1 Key Insights

The development of the CPU Scheduling Simulator provided a better understanding of how operating systems manage processes competing for CPU resources. Through the implementation of different scheduling algorithms, it became clear that scheduling techniques greatly affect system performance, fairness, responsiveness, and efficiency. The project also highlighted the importance of process metrics such as Waiting Time and Turnaround Time in evaluating algorithm performance. By analyzing Gantt Charts and computed metrics, the behavior of each scheduling method became easier to observe and compare.

The project also strengthened understanding of modular programming and algorithm implementation using Python. Separating algorithms into individual modules improved code readability, maintainability, and debugging efficiency. The simulation demonstrated how theoretical scheduling concepts taught in Operating Systems are applied in practical software development and process management.

3.2 Performance Comparison Observations

The implemented scheduling algorithms showed noticeable differences in process handling and overall system efficiency. FCFS scheduling was simple and easy to implement but often resulted in higher waiting times when long processes executed before shorter ones. SJF scheduling reduced average waiting time by prioritizing shorter processes, although longer processes could experience starvation. SRT improved responsiveness further through preemption but increased context switching overhead because processes were interrupted frequently.

Round Robin scheduling provided fairness because all processes received equal CPU time using a fixed Time Quantum. This made the algorithm suitable for multitasking and interactive systems. Priority Scheduling improved execution of important tasks by allowing higher-priority processes to execute first, although lower-priority processes sometimes experienced starvation. Priority Scheduling with Round Robin improved fairness while still maintaining process prioritization.

3.3 Practical Applications in Operating Systems

CPU Scheduling Algorithms are highly important in modern operating systems because they help manage multiple running processes efficiently. These scheduling techniques allow systems to maximize CPU utilization, improve responsiveness, and maintain stable multitasking environments. Scheduling methods are commonly used in desktop computers, mobile devices, servers, and embedded systems where multiple applications and services execute simultaneously.

The project demonstrated how scheduling directly affects user experience and system performance. Efficient scheduling reduces waiting times, prevents unnecessary CPU idle periods, and improves process execution flow. Understanding CPU scheduling is therefore essential for software developers, computer scientists, and system designers involved in operating system development and performance optimization.

3.4 Challenges Encountered and Solutions

One of the major challenges encountered during development was implementing preemptive scheduling algorithms such as SRT and Round Robin. These algorithms required continuous tracking of process execution, remaining burst time, and queue updates. Managing accurate Gantt Chart generation and process timing calculations also required careful debugging because incorrect time handling produced inaccurate Waiting Time and Turnaround Time results.

Another challenge involved handling CPU idle periods correctly. Early versions of the simulator skipped idle intervals instead of displaying them in the Gantt Chart. This issue was resolved by explicitly adding IDLE states whenever no process was available for execution. To simplify debugging and improve readability, the program was organized using modular programming where each scheduling algorithm was placed in a separate file with dedicated helper functions.

References

GeeksforGeeks. (2025). *CPU scheduling in operating systems*. GeeksforGeeks. https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/

GeeksforGeeks. (2025). *Round Robin scheduling in operating system*. GeeksforGeeks. https://www.geeksforgeeks.org/round-robin-scheduling-in-operating-system/

GeeksforGeeks. (2025). *Shortest remaining time first scheduling algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/shortest-remaining-time-first-preemptive-sjf-scheduling-algorithm/

Python Software Foundation. (2025). *Python documentation*. https://docs.python.org/3/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.

TutorialsPoint. (2025). *Operating system process scheduling*. TutorialsPoint. https://www.tutorialspoint.com/operating_system/os_process_scheduling.htm

Repository Link

https://github.com/JSCode-1/Cabrera_BSCS-3B_OS_CaseStudy

Source Code

utils.py

```
# Calculate average waiting time and turnaround time for all processes
def calculate_averages(processes):
    # Guard clause: exit early if no processes
    if not processes:
        return 0.0, 0.0

    # Loop through all processes to sum waiting times
    avg_wt = sum(p.waiting_time for p in processes) / len(processes)
    avg_tat = sum(p.turnaround_time for p in processes) / len(processes)

    return avg_wt, avg_tat

# Print a table showing process details: PID, Arrival Time, Burst Time, Priority, Waiting Time, Turnaround Time
def print_table(processes):
    print("\nPID\tAT\tBT\tPR\tWT\tTAT")

    # Loop: iterate through each process and print its details
    for p in processes:
        print(f"{p.pid}\t{p.arrival_time}\t{p.burst_time}\t{p.priority}\t{p.waiting_time}\t{p.turnaround_time}")

# Combine consecutive entries in the Gantt chart if they belong to the same process
def compress_gantt(gantt):

    # Guard: return empty list if no gantt data
    if len(gantt) == 0:
        return []

    compressed = []
```

```
    current = gantt[0]

    # Loop: check each item after the first to see if it merges with current
    for i in range(1, len(gantt)):

        next_item = gantt[i]

        # Condition: same process AND next starts when current ends (mergeable)
        if current[0] == next_item[0] and current[2] == next_item[1]:

            # Merge: extend current to next item's end time
            current = (
                current[0],
                current[1],
                next_item[2]
            )

        else:
            # No merge: save current and move to next
            compressed.append(current)

            current = next_item

    compressed.append(current)

    return compressed
```

process.py

```
# Represents a process with scheduling information
class Process:
```

```
    def __init__(self, pid, arrival_time, burst_time, priority=0):
        # Initialize process with ID, arrival time, burst time, and priority
        self.pid = pid
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.priority = priority

        self.remaining_time = burst_time

        self.waiting_time = 0
        self.turnaround_time = 0
```

gantt.py

```
# Print a visual Gantt chart showing process execution timeline
def print_gantt(gantt):

    # Guard: no data to display
    if not gantt:
        print("No Gantt data available.")
        return

    print("\nGantt Chart:")

    # Loop 1: print process names across the top row
    for item in gantt:
        print(f"| {item[0]} ", end="")

    print("|")

    # Print first time value
    print(gantt[0][1], end="")

    # Loop 2: print end times for each item along bottom
    for item in gantt:
        print(f"    {item[2]}", end="")

    print()
```

main.py

```
from process import Process
from gantt import print_gantt
from utils import print_table

from algorithms.fcfs import fcfs
from algorithms.sjf import sjf
```

```
from algorithms.srt import srt
from algorithms.round_robin import round_robin
from algorithms.priority import priority_scheduling
from algorithms.priority_rr import priority_rr

# Read and validate an integer input from the user
def read_int(prompt, min_value=None, max_value=None):
    # Loop: keep asking until valid input received
    while True:
        try:
            value = int(input(prompt))
        except ValueError:
            # Catch: input was not a valid integer
            print("Invalid input. Please enter a valid integer.")
            continue
        except KeyboardInterrupt:
            # Catch: user pressed Ctrl+C
            print("\nInput cancelled.")
            raise

        # Condition: check if value is below minimum
        if min_value is not None and value < min_value:
            print(f"Value must be at least {min_value}.")
            continue
        # Condition: check if value is above maximum
        if max_value is not None and value > max_value:
            print(f"Value must be at most {max_value}.")
            continue

        # All validation passed: return the value
        return value

# Read a non-empty string input from the user
def read_nonempty_string(prompt):
    # Loop: keep asking until non-empty string received
    while True:
        try:
            text = input(prompt).strip()
        except KeyboardInterrupt:
            # Catch: user pressed Ctrl+C
            print("\nInput cancelled.")
            raise

        # Condition: string has content
        if text:
            return text
        # Else: string was empty or only whitespace
        print("Value cannot be empty.")

# Main program: Get process details from user and run selected scheduling algorithm
def main():
    processes = []
```

```
try:
    # Get number of processes
    n = read_int("Enter number of processes (minimum 3): ", min_value=3)

    # Get details for each process
    for i in range(n):
        print(f"\nProcess {i + 1}")

        # Collect process information from user
        pid = read_nonempty_string("Process ID: ")
        arrival = read_int("Arrival Time: ", min_value=0)
        burst = read_int("Burst Time: ", min_value=1)
        priority = read_int("Priority: ")

        processes.append(Process(pid, arrival, burst, priority))

    # Display scheduling algorithm options
    print("\nCPU Scheduling Algorithms")
    print("1. FCFS")
    print("2. SJF")
    print("3. SRT")
    print("4. Round Robin")
    print("5. Priority Scheduling")
    print("6. Priority Scheduling with Round Robin")

    # Get user's algorithm choice
    choice = read_int("Choose algorithm: ", min_value=1, max_value=6)

    # Conditional: route to selected algorithm
    if choice == 1:
        # User selected FCFS
        completed, gantt, avg_wt, avg_tat = fcfs(processes)

    elif choice == 2:
        # User selected SJF
        completed, gantt, avg_wt, avg_tat = sjf(processes)

    elif choice == 3:
        # User selected SRT
        completed, gantt, avg_wt, avg_tat = srt(processes)

    elif choice == 4:
        # User selected Round Robin: get time quantum
        quantum = read_int("Enter Time Quantum: ", min_value=1)
        completed, gantt, avg_wt, avg_tat = round_robin(processes, quantum)

    elif choice == 5:
        # User selected Priority Scheduling
        completed, gantt, avg_wt, avg_tat = priority_scheduling(processes)

    else:
```

```
        # Default: choice == 6, Priority RR: get time quantum
        quantum = read_int("Enter Time Quantum: ", min_value=1)
        completed, gantt, avg_wt, avg_tat = priority_rr(processes, quantum)

    except KeyboardInterrupt:
        # Catch: user interrupted program with Ctrl+C
        print("\nProgram terminated by user.")
        return
    except Exception as exc:
        # Catch: any unexpected error occurred
        print(f"An unexpected error occurred: {exc}")
        return

    # Display results
    print_gantt(gantt)
    print_table(completed)

    print(f"\nAverage Waiting Time: {avg_wt:.2f}")
    print(f"Average Turnaround Time: {avg_tat:.2f}")
    input("\nPress Enter to exit...")

if __name__ == "__main__":
    main()
```

fcfs.py

```
from utils import compress_gantt, calculate_averages

def get_arrival(process):
    return process.arrival_time

# FCFS (First-Come, First-Served): Processes execute in the order they arrive
def fcfs(processes):

    processes.sort(key=get_arrival)

    time = 0

    gantt = []

    # Loop: process each job in arrival order
    for process in processes:

        # Condition: CPU is idle before this process arrives
        if time < process.arrival_time:

            gantt.append((
                "IDLE",
                time,
```

```
            process.arrival_time
        ))

        time = process.arrival_time

    start = time

    end = start + process.burst_time

    process.waiting_time = (
        start - process.arrival_time
    )

    process.turnaround_time = (
        end - process.arrival_time
    )

    gantt.append((
        process.pid,
        start,
        end
    ))

    time = end

gantt = compress_gantt(gantt)

avg_wt, avg_tat = calculate_averages(processes)

return processes, gantt, avg_wt, avg_tat
```

sjf.py

```
from utils import compress_gantt, calculate_averages

def get_shortest(process):
    return process.burst_time

# SJF (Shortest Job First): Process with the shortest burst time executes first
def sjf(processes):

    time = 0

    completed = 0

    n = len(processes)

    gantt = []
```

```
finished = []

# Loop: continue until all processes complete
while completed < n:

    ready = []

    # Loop: find all ready processes
    for process in processes:

        # Condition: process has arrived and is not yet finished
        if (
            process.arrival_time <= time
            and process not in finished
        ):

            ready.append(process)

    # Condition: no process is ready
    if len(ready) == 0:

        gantt.append((
            "IDLE",
            time,
            time + 1
        ))

        time += 1

        continue

    # Select shortest job
    current = min(
        ready,
        key=get_shortest
    )

    start = time

    end = time + current.burst_time

    current.waiting_time = (
        start - current.arrival_time
    )

    current.turnaround_time = (
        end - current.arrival_time
    )

    gantt.append((
        current.pid,
        start,
```

```
            end
        ))

        time = end

        finished.append(current)

        completed += 1

    gantt = compress_gantt(gantt)

    avg_wt, avg_tat = calculate_averages(processes)

    return processes, gantt, avg_wt, avg_tat
```

srt.py

```
from utils import compress_gantt, calculate_averages

def get_shortest_remaining(process):
    return process.remaining_time

# SRT (Shortest Remaining Time): Process with shortest remaining time executes first (preemptive)
def srt(processes):

    time = 0

    completed = 0

    n = len(processes)

    gantt = []

    # Loop: initialize remaining time for each process
    for process in processes:
        process.remaining_time = process.burst_time

    # Loop: continue until all processes finish
    while completed < n:

        ready = []

        # Loop: find all processes ready to run
        for process in processes:

            # Condition: process arrived and still has work
            if (
                process.arrival_time <= time
                and process.remaining_time > 0
```

```
        ):

            ready.append(process)

    # Condition: no process ready, CPU is idle
    if len(ready) == 0:

        gantt.append((
            "IDLE",
            time,
            time + 1
        ))

        time += 1

        continue

    # Select process with shortest remaining time
    current = min(
        ready,
        key=get_shortest_remaining
    )

    start = time

    current.remaining_time -= 1

    time += 1

    gantt.append((
        current.pid,
        start,
        time
    ))

    # Condition: process is now complete
    if current.remaining_time == 0:

        completed += 1

        current.turnaround_time = (
            time - current.arrival_time
        )

        current.waiting_time = (
            current.turnaround_time
            - current.burst_time
        )

gantt = compress_gantt(gantt)

avg_wt, avg_tat = calculate_averages(processes)
```

```
    return processes, gantt, avg_wt, avg_tat
```

round_robin.py

```
from collections import deque

from utils import compress_gantt, calculate_averages

# Round Robin (RR): Each process gets a fixed time slice (quantum); unfinished processes go to the back of the queue
def round_robin(processes, quantum):

    time = 0

    completed = 0

    n = len(processes)

    gantt = []

    queue = deque()

    added = []

    # Loop: initialize remaining time for each process
    for process in processes:
        process.remaining_time = process.burst_time

    # Loop: continue until all processes complete
    while completed < n:

        # ADD ARRIVED PROCESSES - Loop: check for new arrivals
        for process in processes:

            # Condition: process arrived and not yet added to queue
            if (
                process.arrival_time <= time
                and process not in added
            ):

                queue.append(process)

                added.append(process)

        # Condition: queue is empty, CPU is idle
        if len(queue) == 0:

            gantt.append((
```

```
        "IDLE",
        time,
        time + 1
    ))

    time += 1

    continue

current = queue.popleft()

start = time

execute = quantum

# Condition: remaining time is less than quantum
if current.remaining_time < quantum:
    execute = current.remaining_time

current.remaining_time -= execute

time += execute

gantt.append((
    current.pid,
    start,
    time
))

# CHECK NEW ARRIVALS - Loop: check for arrivals during execution
for process in processes:

    # Condition: process arrived and not yet added
    if (
        process.arrival_time <= time
        and process not in added
    ):

        queue.append(process)

        added.append(process)

# Condition: process still needs CPU time
if current.remaining_time > 0:

    queue.append(current)

else:
    # Process is complete

    completed += 1
```

```
        current.turnaround_time = (
            time - current.arrival_time
        )

        current.waiting_time = (
            current.turnaround_time
            - current.burst_time
        )

    gantt = compress_gantt(gantt)

    avg_wt, avg_tat = calculate_averages(processes)

    return processes, gantt, avg_wt, avg_tat
```

priority.py

```
from utils import compress_gantt, calculate_averages

def get_priority(process):
    return process.priority

# Priority Scheduling: Process with the highest priority (lowest number) executes first
def priority_scheduling(processes):

    time = 0

    completed = 0

    n = len(processes)

    gantt = []

    finished = []

    # Loop: continue until all processes complete
    while completed < n:

        ready = []

        # Loop: find all ready processes
        for process in processes:

            # Condition: process has arrived and is not finished
            if (
                process.arrival_time <= time
                and process not in finished
            ):
```

```
            ready.append(process)

        # Condition: no process ready, CPU is idle
        if len(ready) == 0:

            gantt.append((
                "IDLE",
                time,
                time + 1
            ))

            time += 1

            continue

        # Select process with highest priority (minimum priority number)
        current = min(
            ready,
            key=get_priority
        )

        start = time

        end = time + current.burst_time

        current.waiting_time = (
            start - current.arrival_time
        )

        current.turnaround_time = (
            end - current.arrival_time
        )

        gantt.append((
            current.pid,
            start,
            end
        ))

        time = end

        finished.append(current)

        completed += 1

    gantt = compress_gantt(gantt)

    avg_wt, avg_tat = calculate_averages(processes)

    return processes, gantt, avg_wt, avg_tat
```

priority_rr.py

```
from collections import deque

from utils import compress_gantt, calculate_averages

# Priority Scheduling with Round Robin: Processes grouped by priority; each group uses round robin with time quantum
def priority_rr(processes, quantum):

    time = 0
    completed = 0
    n = len(processes)
    gantt = []

    # Loop: initialize each process
    for process in processes:
        process.remaining_time = process.burst_time
        process.waiting_time = 0
        process.turnaround_time = 0

    # Get all unique priority levels sorted
    priorities = sorted({process.priority for process in processes})
    # Create a queue for each priority level
    queues = {priority: deque() for priority in priorities}
    # Track which processes have been added to queues
    added = {priority: set() for priority in priorities}

    # Function to add ready processes to their priority queues
    def add_ready_processes():
        # Loop: check all processes for readiness
        for process in processes:
            # Condition: process arrived, has work, and not queued
            if (
                process.arrival_time <= time
                and process.remaining_time > 0
                and process.pid not in added[process.priority]
            ):
                queues[process.priority].append(process)
                added[process.priority].add(process.pid)

    add_ready_processes()

    # Loop: continue until all processes complete
    while completed < n:
```

```
    current_priority = None

    # Loop: find highest priority queue with ready processes
    for priority in priorities:
        # Condition: this priority queue has processes
        if queues[priority]:
            current_priority = priority
            break

    # Condition: no processes ready, CPU is idle
    if current_priority is None:
        gantt.append(("IDLE", time, time + 1))
        time += 1
        add_ready_processes()
        continue

    current = queues[current_priority].popleft()
    start = time
    # Determine execution time: min of quantum or remaining time
    execute = quantum if current.remaining_time >= quantum else current.remaining_time

    current.remaining_time -= execute
    time += execute
    gantt.append((current.pid, start, time))

    add_ready_processes()

    # Condition: process still has work remaining
    if current.remaining_time > 0:
        queues[current_priority].append(current)
    else:
        # Process is complete
        completed += 1
        current.turnaround_time = time - current.arrival_time
        current.waiting_time = current.turnaround_time - current.burst_time

gantt = compress_gantt(gantt)
avg_wt, avg_tat = calculate_averages(processes)

return processes, gantt, avg_wt, avg_tat
```